Measuring Client Device Behavior on the Modern Web

Abhinav Jauhri
Carnegie Mellon University
ajauhri@cmu.edu

Erik Reed
Carnegie Mellon University
erikreed@cmu.edu

ABSTRACT

In recent years, there has been a proliferation of fast Javascript compilers of modern browsers, faster internet speeds, and ubiquity of streaming content and dynamic updates. As a consequence, the demand on users' browsers has increased, and for a large population of the world the internet has become one of the mosed used features on a mobile or desktop. For this reason, we are curious to understand the client-side usage pattern of resources like network and the client's device while the client accesses the internet. We crawl more than 1000 of the most popular sites and perform a series of benchmarks, analyzing browser activity during and after page load. We also analyze some popular tasks performed by clients like uploading photographs, watching videos, group chat to name a few. With this measurement study we hope to uncover interesting statistics related to how operating system and device (e.g. mobile vs desktop machine) affect network activity, amount of system calls, and other metrics like number of concurrent connections. Such pattern would be helpful to model different devices and study their power usage pattern [10]. We also expect this information to be useful to designers of mobile operating systems, browser and web developers, server administrators, and potentially network administrators and internet service providers.

1. INTRODUCTION

Over the years, operating systems have become in principle similar to distributed systems. For this transformation, analyzing distributed file systems [5, 3], and fault-taulerant systems [7] have become imperative for system researchers. Lately, major internet companies like Google, Facebook, Amazon, Twitter and many others have also dedicated research groups to develop in-house systems [1, 2, 8] which billions of its users leverage to attain low latency in data lookups; analytics over huge datasets; secure storage, to name a few features. More importantly, these systems are used by all of us on laptops, mobile phones, tablets, which are being designed to become more efficient in consumption of power, but at the same time more intelligent. By intelligent, we assert that they are communicating more often than ever before with a lot of other machines over the internet in a short span of time. Therefore, majority of the usage of the underlying OS is for the internet and its applications [11].

Here, we propose to study the network interactions, and local utilization of hardware resources by applications termed as *Cloud Applications (CA)*. Such applications rely on remote servers for state information. Caching for CAs is almost redundant. The term *Local Application (LA)* is reserved for applications/softwares which have been analyzed previously [4] and are not subject of our analysis.

Metric	Mobile	Desktop
Mean number of active connections	30.7	44.3
Mean total syscalls	$3.18 \cdot 10^5$	$7.00 \cdot 10^5$
Mean network activity (received)	.48 MB	.99 MB
Mean network activity (sent)	.044 MB	.071 MB

Table 1: A high level comparison of web pages viewed on a mobile device versus those viewed on a desktop browser.

As an overview, we noted that for sampling of websites on the internet, the difference between websites on a mobile device and desktop web browser are substantial. Table 1 shows a summary of these results, which were collected by crawling 1066 popular websites and waiting idly on the page for two minutes with no user activity.

Operations performed by a CA are similar to any LA, but the underlying sequence of steps are different. For example, a read for a CA would fetch data from a remote source (in rare instances the data may be static, in which case a remote fetch is not necessary). Contrary to a LA, which would first try fetching data from the memory, and if that fails, the operation is transferred to the disk.

CAs are currently used by millions of users via web browsers for online document processing, electronic mail access, instant chat, photo editing, reading, storage and many more utilities. On mobile devices, these CAs have dedicated software packages which also rely on remote access of data. In other words, all CAs have state information which is not available locally in the device of the user. For decades researchers have studied read and writes (I/O traces) to the local memory and disk. We would like to address what network data and OS resource patterns are observed which working on CAs specifically. It is pertinent for future systems on users' devices which will be thin i.e. they will require less local hardware resources and less complex softwares; to be developed in accordance with such patterns of network and OS interactions CAs maintain and, importantly, work at optimizatizing power consumption.

2. NOVELTY

The novely of our project is manifold. First, analyzing web operating systems is different from traditional operating systems as they continuously interact with remote machines for state information. Such statistics provide ways of optimizing the overall number of interactions between a remote server and client. Moreover, it is also important for the lightweight operating system to optimally utilize the limited hardware resources like memory and processor cycles.

Lots of research on power modeling for devices aim to collect fine grained data about the device utilization. As stated in [10], collection of utilization of a device's hardware resources is not enough to model the power consumption. For a more finer and accurate modeling, it is pertinent to look at the pattern of system calls. Each system call may result in change of power state of a device. For example *open* may not result in utilization a hardware resource but it does change the state of power consumption of the device. Specifically, we would like to know how many system calls are used while websites are opened or when a client performs heavy workload tasks like uploading photographs on the internet.

Other work like in [6] looks at users' activity. The study and anaysis of structures of multiple social systems has been highlighed in [9]. These measurements are good for improving the client experience on the internet or evolving existing internet applications like Gmail, Facebook, Twitter, or even browsers. Eventually, our measurement may also lead to this but the scope of this paper is limited to collection of statistics related to how operating system and device affect network activity, amount of system calls, and many other expensive tasks.

3. EXPERIMENT PROCEDURE

Two machines were used to run the experiments in parallel. Both machines had similar hardware configuration: Intel Core i5 processor and 8GB physical memory. The software packages used on both machines were: Ubuntu OS 13.04, Linux kernel v3.5.0-26, System Tap v1.7, and Firefox v20.

Using the linux kernel in debug mode and scripts written using the Linux utility Systemtap, we collected the following: all system calls made, all packet activity, and all active connections made by the browser. The syscall and packet data is stored on a per-call basis. That is, each data point includes a timestep and results stream in. In contrast, the active connection data is made periodically: at approximately four times a second, the active number of connections is measured.

3.1 Crawler Collection

The crawler is a Python program that controls the Firefox browser. We use Firefox because it is open source and easier to control than other browsers (e.g. Chrome, Internet Explorer). The crawler is relatively simplistic; it performs no interactions with the page, only navigating to URLs and waiting. While this occurs, we collect metrics (see Section 3.3 for details) in the background.

The websites to crawl were obtained via the Alexa most popular sites index[1]. Alexa, a subsidiary of Amazon, alleges that it tracks 30 million sites worldwide and relies on the browsing activity of "millions of users". The rank of a website is determined by the number of unique visitors in the past three months. We used the Alexa rankings collected in February 2013. We then iterated through the list in descending order.

In addition to measuring system calls and number of connections, we collect various page timing metrics (e.g. time for: page load, DNS lookup, Javascipt evaluation time, DOM load completion). Some domains in the Alexa rankings have no content on their top level domain; a good example is `http://googleusercontent.com`, which hosts all Gmail attachments, but can't be visited directly by web users. To prune sites like this, we use page timing metrics to test if a page actually has content. We disregard the results of a site if any of the conditions are met:

- The page loads instantly (in the case of blank pages).
- The page never loads (in the case of sites that are down or unavailable for some other reason, or when a connection drop occurs midway through page load).
- An infinite loop or continuously blocking Javascript activity occurs, breaking the collection of page timing results, which evaluates Javascript. In this case, the browser is also forcefully terminated via SIGTERM instead of SIGKILL.

To simulate a mobile device, we used a modified HTTP header to specify and mobile device user agent (UA) in order to test how sites require different resources depending on the device used. Consequently, each site was crawled two times: once with a standard, Firefox UA, and another time with a mobile device UA[2].

To automate the browser and perform crawling, we use the browser automation tool Selenium (`http://seleniumhq.org/`). At a high level, our crawler[3] performs the following.
For each site, with and without a mobile user agent:

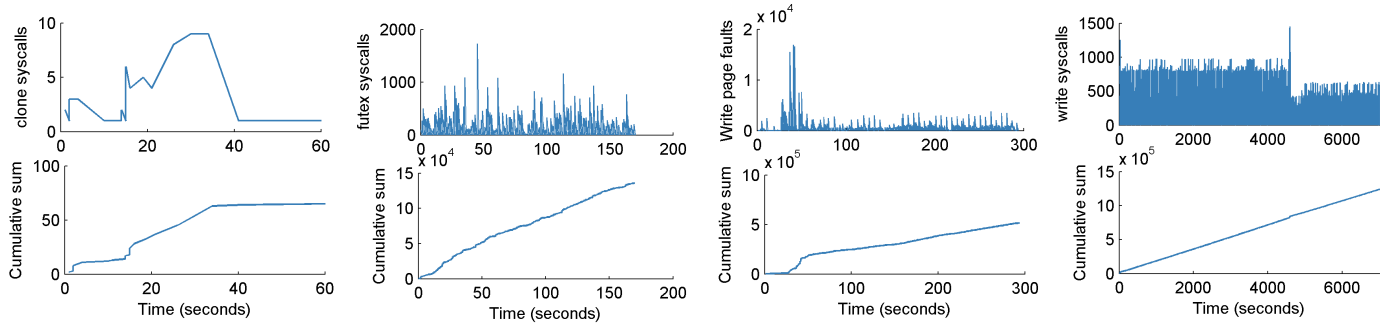
1. Open a new Firefox session (no data/cache/cookies saved) and wait for it to load.
2. Start SystemTap and connection monitoring scripts.
3. Navigate to website URL.
4. Wait for 2 minutes (with no interactions).
5. Stop SystemTap and connection monitoring, then quit Firefox.

We chose to fully quit and restart Firefox to simulate indentical conditions between each site crawled. Additionally, pages that attempted to open new windows or tabs (e.g. popup ads) were allowed to do this, but all new Firefox windows and processes were terminated.

[1] `http://www.alexa.com/topsites`

[2] In particular, we use the UA of the device HTC Desire using Android version 2.3.

[3] All source code is available at `http://git.io/712`



(a) Uploading 5 documents (b) Opening 15 documents (c) Viewing 400 photos (d) Uploading 400 photos

Figure 1: Notable syscall activity during manual activities. Documents were uploaded and viewed via Google Docs (left: a,b) and photos were viewed and uploaded via Facebook (right: c,d).

3.2 Manual Collection

All manual tasks were performed by a real user and were not automated as in the case of the crawler. The time duration for any manual task was not fixed. Each client was allowed to run the task until its successful completion. The desktop machine used to carry out the task was not allowed to perform any other jobs other than the client's task. For each of the manual task a fresh session of Firefox was opened.

The following tasks were performed manually:

- *Uploading photographs*[1] - 248 photographs of 6.8MB size each and 152 photographs of 2.5MB size each were uploaded on a popular social networking site Google+.
- *Video Chat*[1] - For 3 minutes the desktop machine of the client initiating the video chat was monitored. The video chat was carried out using Google video chat feature with Google Talk. There were just two people in the video chat including the initiator.
- *Opening documents on Google Docs*[1] - 15 existing documents were opened from a client's Google drive account.
- *Watching a video* - A 3 minute video was watched on a popular video streaming site youtube.com
- *Uploading files*[1] - Five documents (Microsoft Word) were uploaded on Google Drive. Each document was about 800KB.

During these activities, metrics (see Section 3.3) were collected using the same tools as during the crawling procedure.

The quantitative value for each experiement is similar to the ones mentioned in [4] for future comparisons.

3.3 Metrics Collected

Using a script read into System Tap, we measured the following statistics for each website accessed by the crawler and manual task performed:

- Number of read and write page faults over time.
- Number of bytes received and sent over time.
- Number of sockets created and closed over time.
- Number of threads created over time.
- Number of context switches over time.
- Number of asynchronous writes over time.
- Number of different system calls over time.

To measure the number of connections over time, we used the program *netstat*[4]. The number of connections reported are already connected/active, or "established" in netstat jargon, and this number includes both TCP and UDP connections.

Lastly, we collect measurements from the browser itself by evaluating a Javascript built-in *Performance Timing*[5] to collect the following metrics:

- Time for DNS lookup.
- Time for initial connection.
- Time for page response (after the intial connection is made).
- Time for redirect (if any).
- Time for dynamic object model (DOM) load.

All metrics were collected in the context of the browser and its child processes only. This was done by isolating the PID(s) of the browser, and pruning all the syscalls captured by SystemTap based on an equality check of each processes' PID and PPID.

4. RESULTS

This section will show our measurement results for crawling the top 1066 sites of the Alexa list of most popular sites and for the five manually collected activities (see Sections

[1]For this task, the client's signing into the Google account was also recorded

[4]http://linux.die.net/man/8/netstat

[5]http://www.w3.org/TR/navigation-timing/sec-navigation-timing-interface

3.2 and 3.3). To organize the results, we structure them by starting with manual collection system calls, followed by crawler connections and system calls.

4.1 Manual Collection

For manual tasks, we show some interesting as well as obvious results. In Figure 1a, observe that even simple tasks like uploading PDF files requires a substantial number of *clone* system calls. *clone* system call, used to create a child process, may require a substantial change in the state of power consumption by the device. In Figure 1b, fact that any online (shared) document has to be maintained to its most latest copy around, the number of system calls for *futex* are a lot in magnitude.

In Figure 1c, while viewing photographs not many *write* system calls are expected as in the case of uploading photographs, and it is made certain by the results. In Figure 1a, when uploading photographs the frequency of *write* system calls corresponds to the size of the photograph being uploaded. The sudden fall in the frequency after 4000 seconds is due to the same fact that the photographs were smaller in size as compared to the previous batch uploaded (see Section 3.2).

4.2 Crawler Collection

This section analyzes the data obtained by crawling 1066 popular websites. The results obtained via crawling totaled 274MB of data compressed, the majority being syscalls. The data for each website was treated as a time series; for each time interval, or bin, a number of syscalls were made and the number of active connections was collected multiple times a second; for each bin, these results were aggregated by summing the syscalls and computing the mean of the number of connections. We used 5 seconds to be our bin size in aggregating results and for visualization.

After parsing the syscall data produced by the crawler, we noted 113 unique syscalls were made. Combined with the metrics of Section 3.3, there are 123 total features being collected and aggregated for each bin. That is, for each time interval, there are 123 data points corresponding to syscall, connection, network, and various other features. With a bin size of 5 and 150 seconds of time per site, there are 30 bins for each feature for each site. Consequently, the entirety of the data collected can be reduced to a matrix with dimensionality $30 \times 123 \times 1066$ with ≈ 4 million data points. Because of the scale of the data, we show only a select set of features that were particularly interesting or unexpected. Notably, we exclude performance metrics such as page load time.

First, network metrics (connections and network activity) will be shown, followed by system calls.

4.2.1 Connections and Network Activity

As shown in Table 1, mobile devices show a near 2x decrease in both sent and received network activity compared to a desktop browser UA. Figure 2 shows the network activity (where the cumulative sums form the results of Table 1 network activity). There is an initial jump during and after page load of network activity, reaching near zero (however

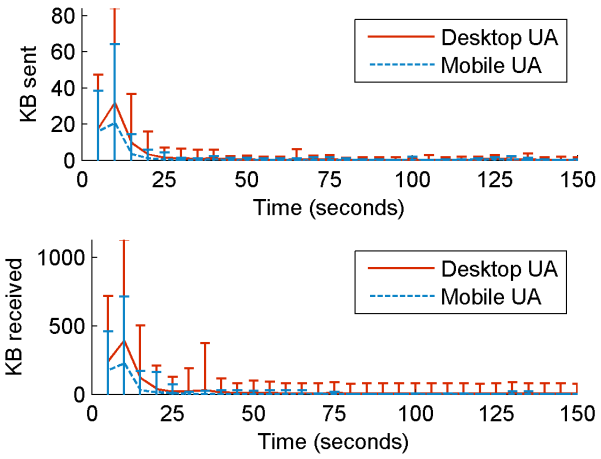


Figure 2: Network activity *rates* over time for data sent (top) and data received (bottom). Each data point corresponds to the summation of network activity in a 5 second interval.

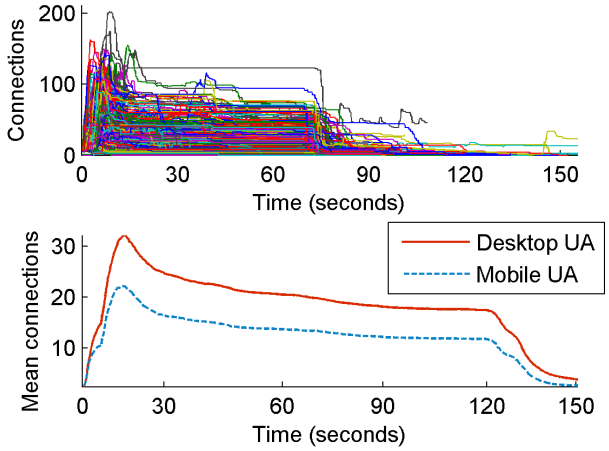


Figure 3: The individual connections (top) and mean number of connections (bottom) over all of the crawled sites.

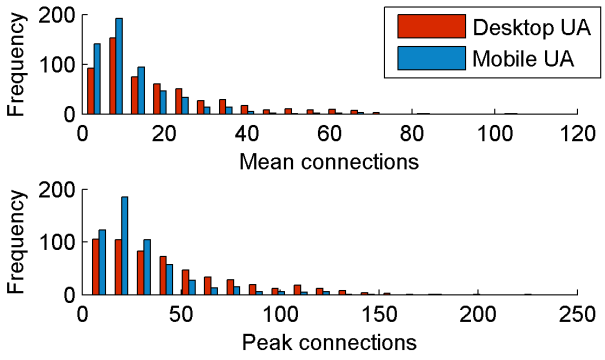


Figure 4: The mean number of connections (top) and peak/max number of connections (bottom) throughout the 150 second timespan for each of the sites crawled.

non-zero) after 25 seconds. This reveals that, on average, 25 seconds must elapse before the majority of the page load is complete. This is counterintuitive, as visually sites will often appear loaded, but continuously cache content for later, anticipated use. The use of mobile UA doesn't reveal a change in this pattern.

Also of interest is the non-zero network activity after 50 seconds; from Figure 3, we note a large number of connections (25 and 17 for desktop and mobile UA, respectively) that remain after the initial page load. While the network activity drops substantially by 25 seconds, the connections show a relatively minor decrease. The majority of these connections are presumed to be analytics software that tracks user activitites. There is an unexpected drop in number of connections near 130 seconds. We estimate this drop is due to either a TCP or UDP timeout, potentially caused by the network used to perform crawling (Carnegie Mellon University). For example, the UDP timeout for *dd-wrt* routers defaults to 120 seconds.

Visualizing the connection for each website as a histogram in Figure 4, we note that the spread (i.e. standard deviation) of the mean and peak number of connections using a mobile UA was smaller. All outliers using large numbers of connections (maxing out at 103 mean and 225 peak) used the desktop UA. However, for lower numbers of connections, the difference in distributions was much smaller.

4.2.2 System Calls

As mentioned earlier that capturing the system call pattern is essential for accurate modeling of power consumption. We summarized in Table 1 that the number of system calls just made by the browser and its child processes while opening CAs is a lot in magnitude. It is also important to understand how the pattern is different for each system call on each CA.

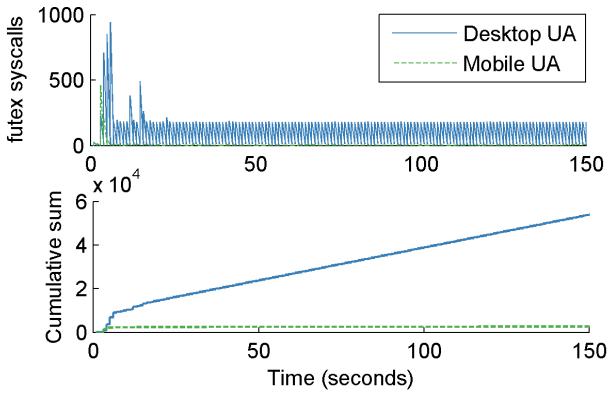
For all our experiements we observed about 115 unique system calls and recorded their frequncies over time. To summarize our results here, we will only show some of them aggregated over 1066 CAs (Figure 6)

Although for Figure 6a and Figure 6d, the pattern in graphs are similar but in Figure 6b and Figure 6c it is not the case.

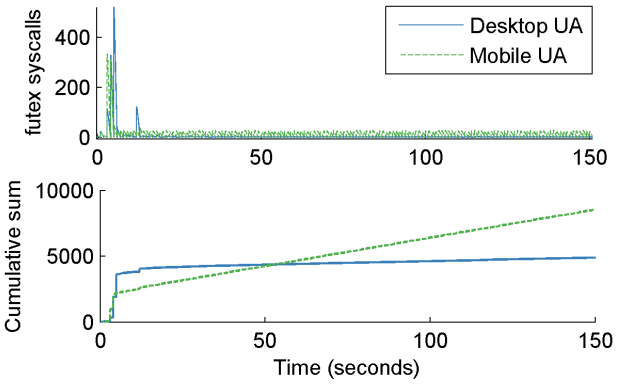
From the peaks in the plots shown in Figure 6b and Figure 6c, it is seen that some sites have a higher utilization of a particular of system call. We show this further in Figure 5. Observe the difference in the frequency for *futex* for different sites. This is valid due to the fact that *amazon.com* has a more dynamic page in comparison to *youtube.com*.

The magnitude of context switches occurring while opening these sites is clearly a lot higher than any of the other system calls. This is an important factor as context switches will not include usage of a physical resource but in terms of power consumption it will be of considerable amount as it involves a lot of overhead.

For future work, we would like to look at system calls which have an impact on the state of power consumption for a device and then look at it on different CAs. In this way, the modeling could be improved.

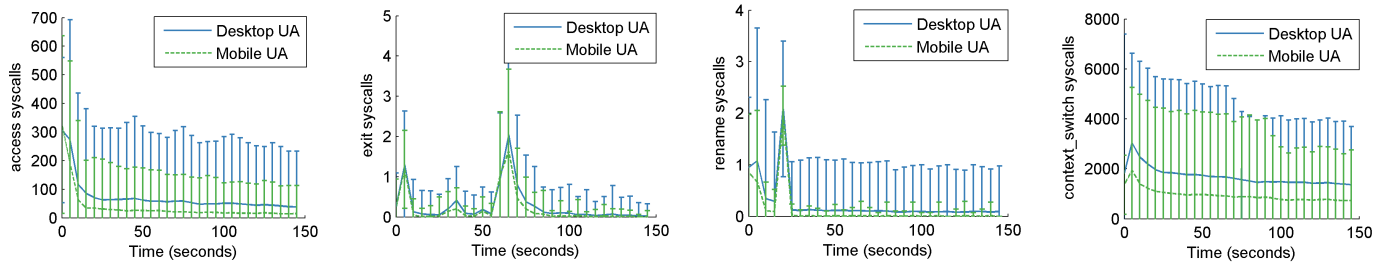


(a) `http://amazon.com`



(b) `http://youtube.com`

Figure 5: A comparison of the *futex* syscall between two sites: Amazon (top) and Youtube (bottom).



(a) Access (b) Exit (c) Rename (d) Context_switch

Figure 6: Aggregated results over 1066 sites for notable syscalls performed. Each data point corresponds to the number of syscalls that occurred in a five second interval. It is interesting to note that each system call has its own pattern and varying behavior based on user agent.

5. CONCLUSION

We observe that the pattern of system calls is not consistent for all sites (e.g. *futex* for `amazon.com` and `youtube.com`). This observation is important because modeling a similar system call pattern for different sites may not give correct results for the consumption of power by the device which was used by the client to access the website. For example, power usage for any user buying items on `amazon.com` will be different from someone who is just viewing videos on `youtube.com`. For a more accurate anaylsis of power consumption, it will be important to group similar sites together and then work on modeling of system/device behavior. The pattern for each system call may also be different from another.

We also noted the difference in the number of open connections for mobile and desktop, and observed significant difference. One may look into optimizing this for desktops as well.

In the future, if such Web OSs are popular and bandwidth-intensive, they can have a significant impact on Internet traffic. Understanding the interactions is critical for developing robust and secure systems.

6. REFERENCES

[1] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

[2] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. In *ACM SIGOPS Operating Systems Review*, volume 41, pages 205–220. ACM, 2007.

[3] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The Google file system. In *ACM SIGOPS Operating Systems Review*, volume 37, pages 29–43. ACM, 2003.

[4] Tyler Harter, Chris Dragga, Michael Vaughn, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. A file is not a file: understanding the I/O behavior of Apple desktop applications. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, pages 71–83. ACM, 2011.

[5] John H Howard, Michael L Kazar, Sherri G Menees, David A Nichols, Mahadev Satyanarayanan, Robert N Sidebotham, and Michael J West. Scale and performance in a distributed file system. *ACM Transactions on Computer Systems (TOCS)*, 6(1):51–81, 1988.

[6] Jeff Huang and Ryen W White. Parallel browsing behavior on the web. In *Proceedings of the 21st ACM conference on Hypertext and hypermedia*, pages 13–18. ACM, 2010.

[7] James J Kistler and Mahadev Satyanarayanan. Disconnected operation in the Coda file system. *ACM Transactions on Computer Systems (TOCS)*, 10(1):3–25, 1992.

[8] Avinash Lakshman and Prashant Malik. Cassandra - a decentralized structured storage system. *Operating systems review*, 44(2):35, 2010.

[9] Alan Mislove, Massimiliano Marcon, Krishna P Gummadi, Peter Druschel, and Bobby Bhattacharjee. Measurement and analysis of online social networks. In *Proceedings of the 7th ACM SIGCOMM conference on Internet measurement*, pages 29–42. ACM, 2007.

[10] Abhinav Pathak, Y Charlie Hu, Ming Zhang, Paramvir Bahl, and Yi-Min Wang. Fine-grained power modeling for smartphones using system call tracing. In *Proceedings of the sixth conference on Computer systems*, pages 153–168. ACM, 2011.

[11] Alex Wright. Ready for a web OS? *Communications of the ACM*, 52(12):16–17, 2009.